

Uygulama: Çok İş Parçacıklı Depo Yönetim Sistemi (Lab4)

Bu uygulamada Java kullanılarak çok iş parçacıklı (multi-thread) bir depo ve envanter yönetim sistemi geliştirilecektir. Uygulamanın amacı, paylaşılan bir depo kaynağı üzerinde birden fazla thread'in aynı anda işlem yapması durumunda oluşabilecek tutarsızlıkları gözlemlemek ve synchronized anahtar kelimesi ile thread-safe (güvenli) bir yapı kurmaktır.

Sınıflar ve Fonksiyonel Açıklamaları

1. Product (Ürün) Sınıfı

Bu sınıf, depoda bulunan her bir ürünü temsil eder.

- **id (int):** Ürünün benzersiz kimlik numarası.
- **name (String):** Ürünün adı.
- **stock (int):** Ürünün mevcut stok miktarı.
- **Product(id, name, stock):** Başlangıç değerlerini atayan yapıcı metot.
- **setStock(int):** Ürün stok miktarını güncelleyen yardımcı metot.

2. Warehouse (Depo) Sınıfı

Bu sınıf, depo adını tutmalı ve ürünleri bir map yapısında saklamalıdır.

- **inventory (Map<Integer, Product>):** Ürün id'si ile Ürün nesnesini eşleyen yapı.
- **maxCapacity (int):** Deponun alabileceği toplam maksimum ürün sayısı.
- **isFull(amount):** Depoya eklenecek yeni miktarın toplam kapasiteyi aşıp aşmadığını kontrol eder.
- **updateAndLog(product, amount, helper):** * Bu metot **synchronized**(senkronize) edilerek thread-safe hale getirilmelidir.
 - Önce isFull ile kapasiteyi, sonra ürünün stok miktarının eksiye düşüp düşmediğini kontrol eder.
 - Şartlar uygunsa stok güncellemesini yapar ve FileHelper üzerinden işlemi dosyaya kaydeder.

3. FileHelper

Bu sınıf gerekli dosya işlemlerinin yapıldığı sınıf olmalıdır.

- **logAction(file, message):** Yapılan işlemleri (Tedarik/Satış), işlem zamanı ile birlikte belirtilen dosyaya ekler (append).
- **saveInventory(file, inventory):** Program sonunda veya başında mevcut envanterin tam listesini dosyaya yazar.

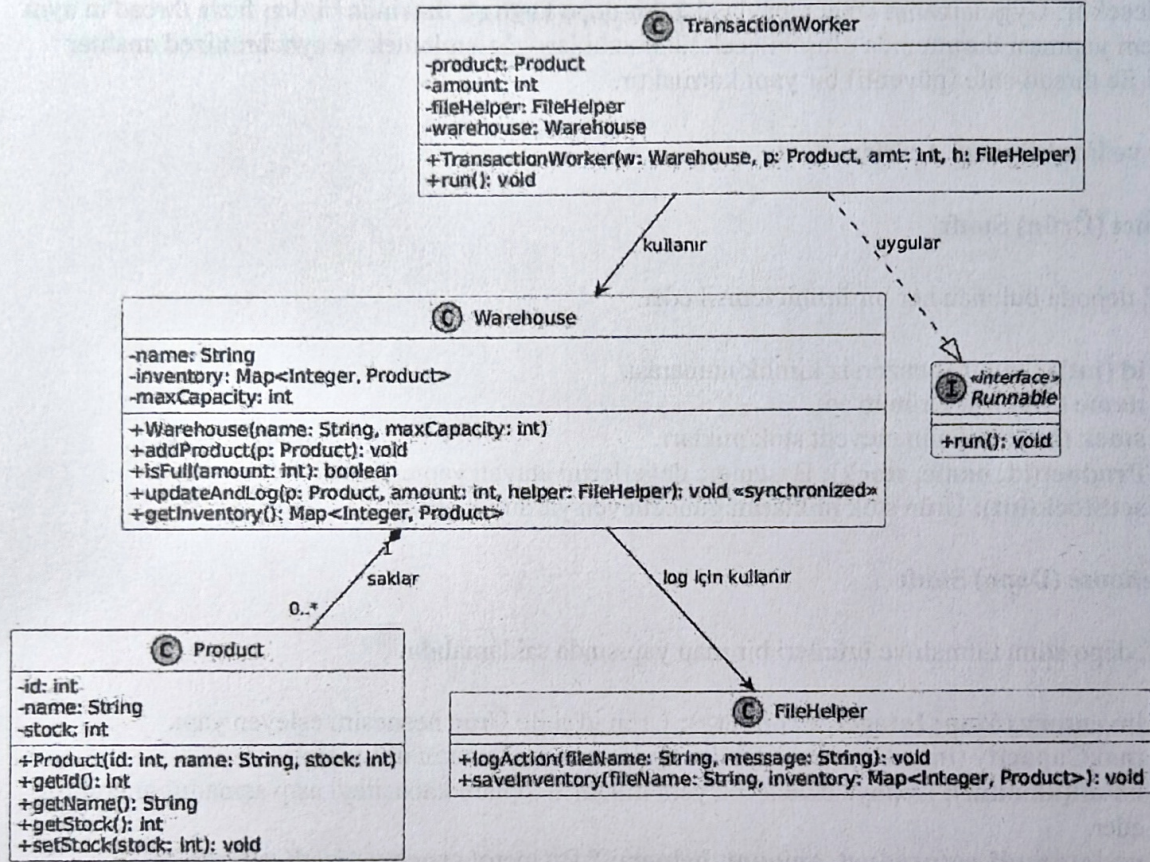
4. TransactionWorker (İşçi) Sınıfı

Runnable arayüzünü uygulamalı ve işlemlerin paralel yürütülmesini sağlamalıdır.

- **run():** Thread başlatıldığında çalışan metottur. Kendisine atanan ürün üzerinde Warehouse.updateAndLog metodunu çağırarak işlemi gerçekleştirir.

NOT1: Her çalıştırmada deponun son durumu farklı olabilir. Önemli olan her thread sıra sıra işlem yapacak olması. Deponun son durumunun farklı olması hangi threadin önce işlem yaptığı ile alakadardır.

NOT2: Thread.currentThread().getName() ile şuan işlem yapan threadin ismini elde edebilirsiniz



101,Islemci,20
102,Ekran Karti,10
103,Anakart,20

Suan da işlem yapan thread: th1
SATIS (-) Ürün: Islemci Miktar: 10 Güncel Stok: 10
Suan da işlem yapan thread: th6
Hata: Depo kapasitesi aşıldı! İşlem yapılamıyor: Islemci
Suan da işlem yapan thread: th5
SATIS (-) Ürün: Ekran Karti Miktar: 10 Güncel Stok: 0
Suan da işlem yapan thread: th3
Hata: Depoda yeterli stok yok! Ürün: Ekran Karti
Suan da işlem yapan thread: th4
SATIS (-) Ürün: Anakart Miktar: 15 Güncel Stok: 5
Suan da işlem yapan thread: th2
TEDARIK (+) Ürün: Islemci Miktar: 20 Güncel Stok: 30

--- Depo Durumu ---

Ürün: Islemci | Stok: 30
Ürün: Ekran Karti | Stok: 0
Ürün: Anakart | Stok: 5